



UNIVERSITÀ
DI TRENTO

Department of Information Engineering and Computer Science

Master's Degree in Information Engineering

Course of Computer Vision

**3D Player Pose Estimation from Multiview Cameras
and MoCap Comparison**

Nicola Cappellaro Riccardo Zannoni

October 31, 2025

1 Introduction

Accurate estimation of 3D human poses from multiview images is a fundamental problem in computer vision, with applications ranging from sports performance analysis to animation and human–robot interaction. Traditional motion capture (MoCap) systems provide highly precise measurements but require specialized equipment and controlled environments, limiting their applicability in real-world scenarios.

The goal of this project is to estimate the 3D pose of a basketball player recorded in the Sanbapolis using a multi-camera RGB setup. Starting from manually annotated 2D keypoints, the player’s 3D skeleton is reconstructed through geometric triangulation. The estimated poses are then compared with ground-truth MoCap data to assess the accuracy of the triangulation. Additionally, we test a modern 2D human pose estimation algorithm (YOLO-Pose) and evaluate its performance.

This work aims to provide a complete multiview pose estimation and evaluation pipeline, including preprocessing, rectification, triangulation, alignment with MoCap data, and accuracy assessment through standard metrics such as Mean Per Joint Position Error (MPJPE).

2 Annotation

The first step of the project consisted in the manual annotation of the basketball player’s body joints from the multiview video recordings captured in the Sanbapolis. Four fixed RGB cameras were used for data acquisition, identified as cameras 2, 5, 8, and 13. Each camera recorded one action performed by the player for 48 frames.

2.1 Annotation Procedure

All annotations were performed using the **Roboflow** platform, which provides an intuitive interface for labeling keypoints in 2D images. Each frame was manually annotated according to the **COCO keypoint format**, consisting of 17 anatomical joints (hips, knees, ankles, shoulders, elbows, wrists, spine, neck, and head). The resulting annotations were exported as a single JSON file, following the standard COCO dataset structure.

2.2 Dataset Structure

The dataset was exported from Roboflow using a single split, `train/`, which contains both the images and the corresponding annotation file. No additional validation or test splits were created, since all available images were used for triangulation and evaluation in later stages.

The final dataset structure is as follows:

```
train/
|-- out2_frame_0001_png.rf.345f188c7aba71764ede395be914924c.jpg
|-- out5_frame_0001_png.rf.126a8c41a7f34f38a891cb870e9123c5.jpg
|-- out8_frame_0001_png.rf.bf4c318a9d42eb9e0e56756cb77d8b76.jpg
|-- out13_frame_0001_png.rf.22bc8a9217e21df53736f86b918c9987.jpg
\-- _annotations.coco.json
```

2.3 Annotation Consistency and Validation

All annotations followed the same joint ordering and labeling convention across every camera view to ensure multi-view consistency. Keypoints that were not directly visible in the camera images were still annotated whenever their approximate position could be inferred, and they were explicitly marked as *occluded* in the dataset. This approach ensured that each frame contained a complete set of joints, allowing a consistent triangulation process across views. Before proceeding to triangulation, all 2D coordinates were verified for completeness and re-exported in a unified COCO-format dataset, ready for rectification and 3D reconstruction.

3 3D Triangulation

After obtaining the 2D annotations from the four camera views, the next step consisted in estimating the 3D position of each joint through geometric triangulation. This process reconstructs the player’s 3D skeleton from multiple synchronized 2D projections, using the intrinsic and extrinsic parameters of each camera.

3.1 Image Rectification

Before triangulation, all camera views were rectified to correct lens distortions and to align the epipolar geometry. The rectification was performed using the calibration parameters provided with the dataset (`camera_calib.json` and `camera_calib_real.json`). It is important to note that the same rectification transformations applied to the raw images must also be applied to the corresponding 2D keypoints, to maintain geometric consistency between views. The rectified images were stored in a dedicated folder (`images_rectified/`) preserving the same file names as the originals.

3.2 Annotation Rectification Script

To ensure geometric consistency between rectified images and the corresponding 2D annotations, a dedicated Python script was developed to apply the same undistortion and rectification transformations used for the videos directly to the keypoints and bounding boxes in the COCO annotation files.

The script loads the intrinsic camera parameters and distortion coefficients for each camera (2, 5, 8, and 13) from their respective calibration files. For every image entry in the COCO JSON, the corresponding rectification maps (`map_x`, `map_y`) are generated using the OpenCV function `initUndistortRectifyMap()`. These maps are then used to transform the pixel coordinates of all annotated keypoints and bounding boxes to their undistorted equivalents.

Each keypoint (x,y) is first clamped to image bounds to prevent invalid pixel access, then remapped to the rectified coordinates using the lookup maps. Bounding boxes are similarly rectified by transforming their four corner points and recomputing the enclosing rectangle. The output is a new annotation file (`_annotations.coco.rectified.json`) that preserves the original structure and metadata but contains geometrically corrected keypoints and bounding boxes.

This step guarantees that the 2D annotations perfectly match the rectified image domain, preventing projection inconsistencies during triangulation and reprojection analysis.



(a) Annotated image



(b) Annotated rectified image

Figure 1: keypoints check after rectification

3.3 Triangulation Method

Given 2D keypoints (x_i, y_i) observed by multiple calibrated cameras with projection matrices $P_i = K_i[R_i | t_i]$, 3D joint positions are estimated with a linear Direct Linear Transform (DLT) solved via SVD. For each joint and view we stack the two homogeneous equations

$$(x_i P_3^\top - P_1^\top) \mathbf{X} = \mathbf{0}, \quad (y_i P_3^\top - P_2^\top) \mathbf{X} = \mathbf{0},$$

and recover $\mathbf{X}$ as the right singular vector associated to the smallest singular value, followed by homogeneous normalization. A minimum of two views per joint is required. Unless otherwise stated, we discard keypoints with visibility $v < 1$ (missing) and use visible and occluded points ($v \geq 1$). This DLT approach minimizes an algebraic error; reprojection error is computed afterwards for evaluation.

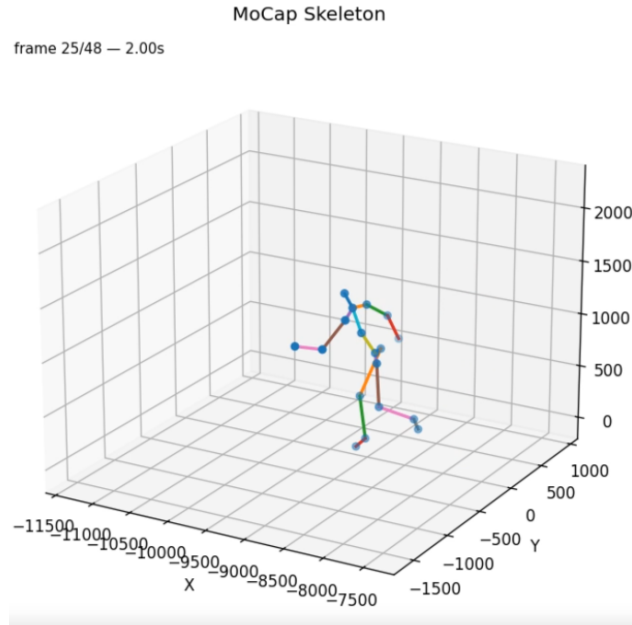


Figure 2: 3D skeleton after triangulation.

3.4 Reprojection and Accuracy Evaluation

To validate the geometric consistency of the 3D reconstruction, the triangulated joints were reprojected onto each camera view using the calibrated intrinsics and extrinsics. Since both the images and the ground-truth

annotations had been rectified (undistorted), reprojections were first computed in the original camera model (with distortion) and then mapped to the rectified domain before comparison. This ensures that the reprojected 2D points are directly comparable to the rectified ground truth without reintroducing lens distortion. We then assembled a COCO-style annotation file containing the reprojected 2D keypoints for every image. For evaluation, we computed the 2D *Mean Per Joint Position Error* (MPJPE) and the *Mean Squared Error* (MSE) between reprojected points and ground-truth annotations:

$$\text{MPJPE}_{2D} = \frac{1}{M} \sum_{j=1}^M \left\| \hat{\mathbf{x}}_j - \mathbf{x}_j^{gt} \right\|_2, \quad \text{MSE}_{2D} = \frac{1}{M} \sum_{j=1}^M \left\| \hat{\mathbf{x}}_j - \mathbf{x}_j^{gt} \right\|_2^2,$$

where $\hat{\mathbf{x}}_j$ denotes the reprojected 2D location and $\mathbf{x}_j^{gt}$ the corresponding ground truth keypoint. Following visibility flags in COCO format, metrics are computed *only on keypoints marked as visible* in the ground truth ($v=2$), while re-projected keypoints are considered available for all joints.

In addition to global MPJPE and MSE, we also report *per-joint* MPJPE to analyze which body parts contribute the most to the error:

- Smallest error: Joint 07 left ankle = **27.41 px**
- Biggest error: Joint 17 left hand = **36.34 px**
- MSE (pixel²): **1311.462**
- MPJPE (pixel): **31.223**

4 Alignment with Motion Capture

The motion capture streams and the multiview RGB footage are not time-synchronized. To enable a fair comparison, we align the triangulated 3D skeletons with the MoCap trajectories by estimating:

- (i) a temporal shift (frame offset), and
- (ii) a spatial transform between the two 3D coordinate systems.

We also adopted the same convention and naming in both annotations by selecting the correspondent joint shared by both sources.

Motion Capture keypoints	Triangulation keypoints	Common Order
1. Hips	1. Hips	1. Hips
2. Spine	2. RHip	2. Spine
3. Spine1	3. RKnee	3. Neck
4. Spine2	4. RAnkle	4. Head
5. Neck	5. RFoot	5. LShoulder
6. Head	6. LHip	6. LEIbow
7. LeftShoulder	7. LKnee	7. LHand
8. LeftArm	8. LAnkle	8. RShoulder
9. LeftForeArm	9. LFoot	9. REIbow
10. LeftForeArmRoll	10. Spine	10. RHand
11. LeftHand	11. Neck	11. LHip
12. RightShoulder	12. Head	12. LKnee
13. RightArm	13. RShoulder	13. LAnkle
14. RightForeArm	14. REIbow	14. LFoot
15. RightForeArmRoll	15. RHand	15. RHip
16. RightHand	16. LShoulder	16. RKnee
17. LeftUpLeg	17. LEIbow	17. RAnkle
18. LeftLeg	18. LHand	18. RFoot
19. LeftFoot		
20. LeftToeBase		
21. RightUpLeg		
22. RightLeg		
23. RightFoot		
24. RightToeBase		

Figure 3: Common mapping for the joints.

In conclusion, the mocap sequence was resampled at a common frame rate prior to alignment. The following sections provide a more detailed explanation.

4.1 Spatial Alignment

We estimated the optimal spatial alignment (rotation, traslation and scaling) using Procrustes/Kabsch: centering both sets, computing the cross-covariance SVD, and extracting $\mathbf{R}$ with determinant correction; $\mathbf{t}$ follows from the centroids, and $\mathbf{s}$ from the ratio of Frobenius norms.

4.2 Temporal Alignment

To achieve temporal alignment between the two sequences, we first considered the *shot* (a characteristic position in the basketball motion) as the baseline reference. Then, we identified the first and last frames of each sequence. The correspondences obtained are as follows:

- Start: frame 1 corresponds to frame 980 of the MoCap sequence
- Shot: frame 42 corresponds to frame 1322 of the MoCap sequence
- End: frame 48 corresponds to frame 1370 of the MoCap sequence

We therefore concluded that the two sequences are temporally misaligned with respect to their frame rates, with a temporal offset corresponding to approximately 8.3 times the frame rate.

4.3 Evaluation Against MoCap (3D Metrics)

Evaluation of Alignment Accuracy

As in the previous step, we evaluated the accuracy of our 3D reconstruction by comparing the aligned triangulated skeleton with the corresponding MoCap data. For each frame pair $(t, \tau(t))$ obtained after synchronization, we compute standard pose estimation metrics such as the **MPJPE** and the **MSE** between corresponding joints and obtained the following results:

- Average MPJPE over all frames: **69.66 mm**
- Median MPJPE over all frames: **69.81 mm**
- Average MSE over all frames: **5767.82 mm²**
- Average RMSE over all frames: **75.27 mm**

5 Step 4: Human Pose Estimation from RGB

In this step we assess whether a modern 2D Human Pose Estimation (HPE) model, the pre-trained **YOLO v11 pose model**, can replace manual keypoints for the multiview triangulation pipeline. We run a single-frame, real-time keypoint detector on each rectified camera stream, evaluate its 2D accuracy against ground-truth (GT) annotations, triangulate the detected keypoints with the same method of Step 2, and finally compare the resulting 3D poses with MoCap.

5.1 Detector and Inference Settings

To ensure compatibility with the MoCap reference, we adapted YOLO detections by (i) filtering frames with multiple people to retain only the player wearing the black suit across all cameras, and (ii) remapping/removing incompatible joints so that the YOLO keypoints match the MoCap joint set. Finally, we merged the per-camera JSONs (cam_2, cam_5, cam_8, cam_13) into a single COCO-style annotation file.

5.2 2D Evaluation w.r.t. Ground Truth

To quantify 2D accuracy, we compare detector outputs with the rectified GT annotations used in Step 1. We compute the overall *2D MPJPE* and *MSE*.

$$\text{MPJPE}_{2D} = \frac{1}{M} \sum_{j=1}^M \left\| \hat{\mathbf{x}}_j - \mathbf{x}_j^{gt} \right\|_2, \quad \text{MSE}_{2D} = \frac{1}{M} \sum_{j=1}^M \left\| \hat{\mathbf{x}}_j - \mathbf{x}_j^{gt} \right\|_2^2.$$

We report per-joint statistics to highlight typical failure modes.

- Overall MPJPE (px): **12.318 px**
- Overall MSE (px²): **388.744 px²**

Per-joint mean Euclidean error (px):

- RHip : 24.053
- RKnee : 10.281
- RAnkle : 8.845
- LHip : 12.172
- LKnee : 10.112
- LAnkle : 18.629

- Head : 13.714
- RShoulder : 10.358
- RElbow : 9.718
- RHand : 8.688
- LShoulder : 8.118
- LElbow : 13.588
- LHand : 11.860

5.3 Triangulation of HPE Keypoints

For each frame, multiview 2D keypoints produced by the detector are triangulated with the same linear DLT/SVD method of Step 2. We discard joints with fewer than two valid views and use visible and occluded keypoints ($v \geq 1$) while ignoring missing ones ($v = 0$). The output is a frame-indexed JSON with 3D joint coordinates, consistent with the earlier pipeline, ready to be compared to access the accuracy of the model

5.4 3D Evaluation w.r.t. MoCap

We align the HPE-based 3D sequence to MoCap using the same procedure of Step 3: temporal offset estimation and spatial Procrustes/Kabsch (similarity). Accuracy is reported with standard 3D metrics:

- Average MPJPE over all frames: **68.90 mm**
- Median MPJPE over all frames: **66.10 mm**
- Average MSE over all frames: **5947.10 mm²**
- Average RMSE over all frames: **75.30 mm**

6 3D Skeleton Visualization on Unreal Engine

In order to visualize the skeletons in Unreal Engine, we first used Blender as a bridge: a script exported two FBX animations, one from the MoCap JSON and one from our manual triangulation JSON. We then imported both FBXs into a blank Unreal Engine project and placed the animations in the same scene, overlapping the players to enable a clear, qualitative side-by-side comparison of the MoCap and triangulated 3D skeletons.

7 Conclusion

We implemented an end-to-end multiview pipeline for 3D player pose estimation: calibration and rectification, triangulation, a 2D reprojection sanity check, and alignment to MoCap (temporal + Procrustes/Kabsch). The pipeline is geometrically consistent (2D MPJPE ≈ 31 px, MSE ≈ 1311 px²) and compares well against MoCap (Avg. MPJPE ≈ 69.7 mm; Median ≈ 69.8 mm; RMSE ≈ 75.3 mm).

In Step 4, we replaced manual keypoints with YOLO v11 detections (after person filtering and joint remapping) and obtained comparable 3D accuracy (Avg. MPJPE ≈ 68.9 mm; RMSE ≈ 75.3 mm), indicating that automatic detections can effectively drive the same triangulation/alignment pipeline. Residual errors mostly stem from imperfect synchronization, calibration noise, and occlusions—especially on distal joints. Finally, exporting both MoCap and triangulated animations to FBX and visualizing them overlapped in Unreal Engine enabled a clear qualitative comparison.

In summary, a carefully calibrated and aligned multiview pipeline—driven by either manual annotations or

YOLO-based detections—can approximate MoCap within a few centimeters, offering a practical and scalable alternative when marker-based capture is not feasible.

Bibliography

- [1] Z. Fang, A. Wang, C. Bu and C. Liu, "3D Human Pose Estimation Using RGBD Camera," 2021 IEEE International Conference on Computer Science, Electronic Information Engineering and Intelligent Control Technology (CEI), Fuzhou, China, 2021
- [2] Q.-Y. Zhou and V. Koltun, "Robust Triangulation under Uncertainty," in *ECCV Workshops*, 2016.
- [3] S. Umeyama, "Least-Squares Estimation of Transformation Parameters Between Two Point Patterns," *IEEE Trans. PAMI*, vol. 13, no. 4, pp. 376–380, 1991.
- [4] W. Kabsch, "A solution for the best rotation to relate two sets of vectors," *Acta Cryst. A*, vol. 32, no. 5, pp. 922–923, 1976.